

LC 686: Repeated String Match

Adobe, Uber

Problem Statement

given 2 strings a, b, need to turn b into a substring of a

allowed operations :- repeat a

o/p: min number of times a needs to be repeated

Example & Structure

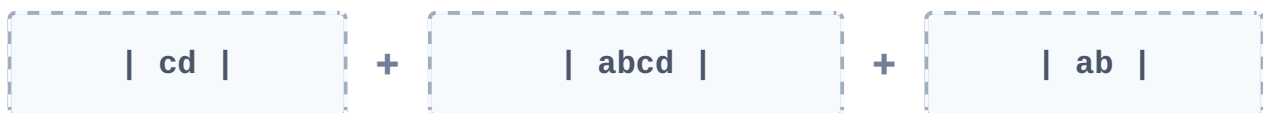
```
a = "abcd"  
b = "cdabcdab"
```

then abcd abcd abcd
 └─── b ──┘

hence **o/p = 3** as a had to repeat itself 3 times

hence the structure of b is something like

suffix of a + 0 or more copies of a + prefix of a



"cyclic repetition order must be maintained"

Solution Logic

- 0 write a infinitely and put B on top.
if b fits completely inside the ∞ string, need to know how many copies of a we need to gen.
- 1 keep adding a copies until the string is at least as long as b

- 2 check if b is present in curr
i.e

```
for (int i = 0; i + b.size() <= curr.size(); i++) {  
    bool same = true; // valid start points  
    for (int j = 0; j < b.size(); j++) {  
        if (curr[i+j] != b[j]) {  
            same = false;  
            break;  
        }  
    }  
    if (same) return cnt;  
}
```

- 3 add another copy of a to check again because b may start in middle

Final Implementation

```
class Solution {
public:
    int repeatedStringMatch(string a, string b) {

        string curr="";
        int ans=0;

        //keep adding a
        //until curr is at least as long as b
        while(curr.size()<b.size()){

            curr+=a;
            ans++;
        }

        //check whether b is inside curr
        if(curr.find(b)!=string::npos){
            return ans;
        }

        //b may start in the middle of one copy of a
        //so one extra copy can complete it
        curr+=a;
        ans++;

        if(curr.find(b)!=string::npos){
            return ans;
        }

        return -1;
    }
};
```